

An intelligent agent with a human-in-the-loop approach to support the specification of software safety requirements: A work in progress

Daniel Pérez-Morera
CITIC, PCI
University of Costa Rica
San José, Costa Rica
daniel.perezmorera@ucr.ac.cr

Abstract—Security requirements specification remains a challenging activity in requirements engineering, particularly because it requires specialized knowledge that is often unavailable in development teams. Recent studies have shown that large language models (LLMs) can support the elicitation of security requirements derived from functional requirements, achieving high coverage but lower relevance compared to human-generated results. In this context, this work proposes an intelligent virtual agent with a human-in-the-loop approach to support users with limited experience in the specification of software security requirements.

This paper reports two related studies conducted at the University of Costa Rica. First, an empirical study (UCR26) evaluated an LLM-based approach for generating security requirements across multiple independent runs. Second, a pilot study (UCR27) involved four participants interacting with an intelligent virtual agent to review, refine, and validate LLM-generated security requirements. The evaluation considered coverage using recall, relevance using precision, and user satisfaction using the System Usability Scale (SUS).

The descriptive results show differences between both conditions. UCR27 presents higher recall values compared to UCR26 runs, while showing lower precision values. The SUS results for UCR27 indicate a mean score of 69.375 across participants. These results are reported as descriptive observations of the evaluated configurations in the studied context.

Index Terms—Security Requirements Engineering, Large Language Models, Human-in-the-Loop, Intelligent Virtual Agents

I. INTRODUCTION

The software development life cycle frequently faces multiple challenges related to requirements specification, and security requirements are often overlooked or addressed only implicitly. Poor management of security requirements can lead to unintended functionality, multiple vulnerabilities, and inefficient use of resources. In many cases, the omission of security requirements is caused by time constraints or a lack of the necessary security knowledge.

However, given this shortage of security specialists, the software engineering community has established a baseline of knowledge through guidelines, standards, and recommendations that support the process of security requirements specification. Under this context, several studies [1]–[5] have explored the use of security patterns and templates by groups

of practitioners for the specification of security requirements. More recently, [6] investigated the replacement of human participants with a large language model. The results of this latter study were promising, achieving high coverage in the identification of security requirements, outperforming most human participants in previous replications. However, the model produced more conservative results in terms of relevance, generating security requirements with lower precision than those produced by humans.

These findings suggest that while LLMs are effective in recovering a broad set of possible security requirements, they still require human supervision to improve the contextual relevance and practical usefulness of the generated results. This limitation motivated the present research, which proposes the incorporation of a human-in-the-loop approach through the creation of an embodied intelligent virtual agent that supports the requirements elicitation process.

Unlike fully automated approaches, the proposed virtual agent allows users to review, reject, refine, and validate the generated security requirements during the elicitation process, combining the coverage capabilities of LLMs with human judgment and domain understanding. In addition to evaluating the quality of the generated requirements through coverage and relevance, this study also analyzes user satisfaction using the System Usability Scale (SUS), since practical adoption depends not only on technical performance but also on usability and user acceptance.

Specifically, the general objective of this research is:

To describe the process of eliciting security requirements implicitly derived from functional requirements using an intelligent virtual agent, with the purpose of exploring the observed coverage, relevance, and user satisfaction throughout the elicitation process from the perspective of a requirements engineer.

Given that the experiment was designed as a pilot study, only one execution was conducted with a convenience sample of four participants. Participants presented heterogeneous levels of computing knowledge, ranging from basic to advanced expertise. This diversity was intentionally preserved to obtain an initial exploratory understanding of how users with different

technical backgrounds interact with the intelligent virtual agent during the security requirements elicitation process.

The study compares a control condition, where security requirements are generated by the LLM without human intervention, and an experimental condition, where participants interact with the intelligent virtual agent using a human-in-the-loop approach. Based on the general objective, the research questions of this study are as follows:

- **RQ1:** How does the observed coverage of security requirements differ between the control group and the experimental group?
- **RQ2:** How does the observed relevance of security requirements differ between the control group and the experimental group?
- **RQ3:** What level of user satisfaction is perceived by participants after interacting with the intelligent virtual agent?

For clarity, we define the following code to refer to the previous empirical study:

- **UCR26:** Empirical study conducted at the University of Costa Rica in 2026. This study evaluated a large language model (LLM) for generating security requirements from functional requirements.

II. BACKGROUND & RELATED WORK

A. Security Objectives

Security objectives are those security characteristics or properties that are desired in a system [7]. There are six main security objectives that define the behavior that must be guaranteed or prevented: (C) Confidentiality, the degree to which the data is disclosed only as intended. (I) Integrity, the degree to which a system or component guards against improper modification or destruction of computer programs or data. (A) Availability, the degree to which a system or component is operational and accessible when required for use. (ID) Identification & Authentication, the need to establish that a claimed identity is valid for a user, process, or device. (AY) Accountability, the degree to which actions affecting software assets can be traced to the actor responsible for the action. (PR) Privacy, the degree to which an actor can understand and control how their information is used. Security objectives of software systems are not only technical aspects (from the perspective of systems development), but also operational and administrative aspects [1], [8].

B. Security Requirements

Security requirements are those that describe the desired security behavior in a system, or are also defined as the constraints of a system, where one of these constraints can implement one or more security objectives [1], [9]. Security requirements engineering (SRE) is the process of obtaining, specifying, and analyzing security requirements for a software system. Today, there are multiple approaches to support SRE processes, some based on standards such as ISO/IEC 5055 and others based on conceptual frameworks and requirements

modeling, based on security objectives, resources, assets, and any risks associated with a system [8], [10], [11].

C. Security Requirements Elicitation

Methods for the elicitation and documentation of security requirements include both conceptual frameworks and requirements modeling approaches. Among framework-oriented methods, SQUARE [11] provides a structured reference model for organizing and coordinating the technical activities and artifacts involved in security requirements engineering (SRE), following a maturity-driven approach. Another framework-based approach conceptualizes security requirements as constraints and employs them to construct satisfaction arguments for validating the satisfaction of security requirements [12].

Modeling-based approaches for security requirements elicitation include misuse cases [13], [14], misuse activities [15], and abuse cases [16]. These techniques capture the attacker's perspective and facilitate the identification of security requirements intended to mitigate potential attack scenarios. In contrast to these modeling-based approaches, our method focuses on the use of security requirements templates as the primary mechanism for identifying security requirements. It is important to note that different approaches to security requirements identification can be applied in a complementary manner to achieve a more comprehensive analysis.

Although current methodologies have been effective, they require expert personnel who understand the security objectives proposed for the system and the respective specification of the security requirements associated with these objectives, and lack detailed advice on how to contextualize the security requirements for a system [3], [4].

D. Security Patterns and Templates

Security requirements can be reused across systems that share common security objectives. Parameterized templates have been proposed to support reusable security requirements [17], [18], although these approaches provide limited guidance on templates applicability. Empirical studies indicate that the use of patterns, catalogs, and other structured security knowledge generally improves the performance of non-experts, with few exceptions [19]–[21]. Building on this line of work, [1] identified six core security objectives and proposed an initial set of security requirements templates that can be mapped to existing security patterns to support requirement elicitation.

E. Virtual Agents for Security Requirements Elicitation

Although security has been a widely studied topic, we have not identified any virtual agent specifically designed to support the elicitation and specification of security requirements. However, we found some related works in which virtual agents assist security analysis processes [22], [23]. One of these works presents a conversational agent that supports users in generating datacenter network configurations from informal and ambiguous requests [23]. The system combines a domain-adapted Rasa NLU pipeline, large language models, and semantic role labeling to interpret user intent, clarify

missing information, and automatically generate machine-executable network configurations. Although this approach addresses requirement elicitation in a security-related domain, its primary focus is network configuration rather than security requirements specification. Another related work proposes a multi-agent system for real-time security analysis of software releases [22]. This approach uses a set of specialized agents responsible for static and dynamic code analysis, anomaly detection, code verification, and architecture validation. The objective is to improve software security assessment by automating the identification of high-risk releases and supporting cybersecurity experts in decision-making. However, the system focuses on post-development security analysis rather than early-stage security requirements elicitation. Although both works employ agents and address security-related problems, neither proposal includes embodied virtual agents with a corporal representation capable of interacting with stakeholders during the requirements elicitation process. This represents an important gap, particularly in contexts where human-like interaction may improve communication, trust, and requirement completeness.

III. METHODOLOGY

This study was designed as a pilot experiment to evaluate the impact of a human-in-the-loop approach in the elicitation of software security requirements using an intelligent virtual agent. The experiment compares the performance of a fully automated large language model (LLM) against the same model supported by human interaction through a virtual embodied agent.

A. Virtual Agent Implementation

The virtual agent was developed using Unity in a 3D environment, where an embodied avatar was created to simulate a natural interaction with the participant. The avatar was designed with a professional appearance similar to that commonly found in software development companies, with the intention of making the interaction feel familiar and realistic¹. This design decision was influenced by embodied conversational agent principles proposed by Cassell [24], where visual embodiment and conversational interaction can improve user engagement and communication.

The agent has access to multiple subsystems required for real-time interaction. For Speech-to-Text (STT), OpenAI Whisper API was used due to its low response latency, which helps maintain a fluid conversation. For Text-to-Speech (TTS), GPT-4o mini TTS was selected because, in addition to its fast audio generation, it provides high-quality and customizable voices. The conversational support provided by the agent was also inspired by relational agent concepts proposed by Bickmore and Picard [25], aiming to create a more supportive and approachable interaction experience. The system was deployed as a web-based application to ensure accessibility for all participants through a standard browser.

¹https://github.com/daperez03/seqreq_agent.git



Fig. 1. Virtual Agent

As shown in Figure 1, the agent displays only the upper half of its body, maintaining a formal and approachable visual design. This professional appearance may also influence user behavior according to the Proteus Effect [26], encouraging participants to interact more attentively during the elicitation process.

B. Interaction Flow

The interaction process is illustrated in Figure 2. First, the participant provides a set of functional requirements to the virtual agent. The agent sends these requirements to the security requirements elicitation model, which generates a corresponding set of security requirements. These generated requirements are then presented to the participant, who can review, reject, refine, or accept them. This interaction enables the human-in-the-loop process, where human judgment complements the automated generation of the LLM.

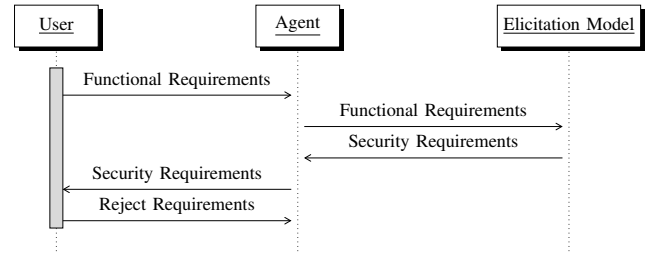


Fig. 2. Security requirements elicitation process

C. Experimental Design

This study was designed as a pilot experiment to evaluate the impact of a human-in-the-loop approach in the elicitation of software security requirements using an intelligent virtual agent. The experiment compares the performance of a fully automated large language model (LLM) against the same model supported by human interaction through a virtual embodied agent.

The independent variable of this study is the use of human intervention during the elicitation process.

- **Control Group:** Security requirements generated by the LLM without human intervention.

- **Experimental Group:** Security requirements generated by the LLM with human interaction through the intelligent virtual agent (human-in-the-loop approach).

The comparison between these two conditions, following a between-subjects design, allows the study to explore the impact of human interaction mediated by the intelligent virtual agent during the security requirements elicitation process. Both conditions rely on the same underlying LLM, ensuring that the core generation capabilities remain constant across the experiment. Therefore, any observed differences in the quality, completeness, clarity, or relevance of the generated security requirements are reported in relation to the presence or absence of the human-in-the-loop interaction provided by the virtual agent.

D. Evaluation Metrics

For this research, *recall*, *precision*, and the *System Usability Scale (SUS)* were used to evaluate coverage, relevance and user satisfaction associated with the security requirements elicitation process. These metrics were selected to assess both the quality of the generated security requirements and participants' perception of the intelligent virtual agent.

To calculate coverage and relevance, the generated security requirements were compared against a predefined security requirements oracle. The comparison was based on the identification of true positives (TP), false positives (FP), and false negatives (FN), defined as follows:

$$TP = |\{x \mid x \in R \wedge x \in O\}| \quad (1)$$

$$FP = |\{x \mid x \in R \wedge x \notin O\}| \quad (2)$$

$$FN = |\{x \mid x \in O \wedge x \notin R\}| \quad (3)$$

where:

- U is the set of all possible requirements.
- $x \in U$ represents an individual requirement.
- $R \subseteq U$ is the set of requirements generated by the model.
- $O \subseteq U$ is the oracle set of requirements.

The results related to coverage and relevance were used to compare the performance of the fully automated LLM approach against the human-in-the-loop approach supported by the virtual agent. Additionally, the satisfaction results were used to evaluate the perceived usability and acceptance of the proposed solution.

To this end, the following metrics were calculated:

- **Coverage:** recall measure with respect to the security requirements templates defined in the security requirements oracle previously developed for the study. Recall measures the proportion of correctly identified security requirements compared to the total number of requirements expected in the oracle.
- **Relevance:** precision measure with respect to the security requirements templates defined in the security requirements oracle previously developed for the study. Precision

determines the proportion of generated requirements that are relevant according to the oracle.

- **Satisfaction:** measured using the System Usability Scale (SUS) proposed by Brooke (1996), which evaluates the perceived usability of the intelligent virtual agent. This instrument measures aspects such as ease of use, usefulness, confidence, and user support during the security requirements elicitation process.

E. Experimental Procedure

The experiment was conducted following these steps:

1) Informed Consent

Participants attended either a virtual or in-person session where the informed consent form was presented. This document explained the objectives of the study, data handling procedures, and the voluntary nature of participation. After reviewing the document, participants signed it to confirm their acceptance.

2) Introduction

A general explanation was provided regarding existing security patterns and previous studies related to security requirements elicitation. After this introduction, participants received a list of functional requirements from which they had to identify and specify the corresponding security requirements.

3) Laboratory Session

The working environment and the intelligent virtual agent were provided to each participant. The virtual agent accompanied the participant during the generation of security requirements. Each participant had a maximum of one hour to complete the task.

4) Satisfaction Survey

Once the laboratory session was completed, participants answered a satisfaction survey based on the System Usability Scale (SUS). This survey aimed to evaluate their perception of the virtual agent. A maximum of 15 minutes was allocated for completing the survey.

5) Closing and Data Storage

Finally, participants were thanked for their collaboration. The collected data were securely stored, ensuring confidentiality and exclusive use for research purposes.

To ensure methodological consistency between the research questions, variables, evaluation metrics, and experimental tasks, a methodological consistency matrix was defined. As shown in Table I, the matrix summarizes the relationship between each research question and the corresponding variables, instruments, and evaluation activities used throughout the experiment. The purpose of this matrix is to provide traceability between the objectives of the study and the procedures applied during data collection and analysis.

IV. THREATS TO VALIDITY

This section discusses the main threats to validity of the proposed pilot study, organized into internal, external, construct, and conclusion validity.

Research Question	Variable	Instrument	Task
How does the observed coverage of security requirements differ between the control group and the experimental group?	Coverage	Recall	Generate and compare security requirements against the oracle templates.
How does the observed relevance of security requirements differ between the control group and the experimental group?	Relevance	Precision	Evaluate the relevance of generated security requirements according to the oracle.
What level of user satisfaction is perceived by participants after interacting with the intelligent virtual agent?	Satisfaction	System Usability Scale (SUS) questionnaire	Participants complete the SUS survey after interacting with the virtual agent.

TABLE I
METHODOLOGICAL CONSISTENCY MATRIX

Several factors may affect the internal validity of the experiment. First, the study relies on a single execution, which limits the ability to control for random variation in LLM outputs and user interaction behavior. Since large language models may produce non-deterministic results, differences in generated security requirements may be partially influenced by stochastic variation rather than solely by the human-in-the-loop intervention.

Second, participants were recruited using a convenience sampling strategy and may have had different levels of prior knowledge in software engineering and security requirements. This heterogeneity can introduce confounding effects, as performance differences may be partially explained by participant expertise rather than the experimental condition.

The generalizability of the results is limited due to the use of a small and non-random sample of graduate students from a single university context. Therefore, the findings may not generalize to professional software engineers or industrial settings.

Additionally, the experimental setting was conducted in a controlled environment using a specific virtual agent implementation and a particular LLM configuration. Different models, prompting strategies, or interaction designs may lead to different outcomes.

Construct validity threats arise from the operationalization of key variables. Coverage and relevance were measured using recall and precision against a predefined oracle of security requirements. However, the quality of this oracle directly impacts the validity of the evaluation, and any incompleteness or subjectivity in its definition may bias the results.

Conclusion validity is limited by the small sample size and the absence of statistical hypothesis testing. Although differences between conditions are reported descriptively, no formal statistical tests were applied to determine significance.

Additionally, inconsistencies in sample reporting (e.g., differences between the intended pilot size and the number of analyzed participants) may affect the reliability of the conclusions and should be clarified in future iterations of the study.

Overall, this work should be interpreted as an exploratory pilot study aimed at identifying preliminary trends in the use of human-in-the-loop virtual agents for security requirements elicitation, rather than providing definitive empirical evidence.

V. RESULTS

This section presents the results obtained from the control condition (UCR26a–UCR26d) and the experimental condition (UCR27), focusing on coverage (recall), relevance (precision), and perceived usability.

A. Security Requirements Quality Results

Table II presents the results for both conditions. The control condition consists of four independent runs (UCR26a–UCR26d), while the experimental condition corresponds to UCR27, which incorporates the human-in-the-loop virtual agent.

ID	Group	Coverage	Relevance
UCR26a	Control	0.715	0.634
UCR26b	Control	0.727	0.639
UCR26c	Control	0.718	0.636
UCR26d	Control	0.708	0.627
UCR27	Experimental	0.821	0.548

TABLE II
COMPARISON OF SECURITY REQUIREMENTS QUALITY BETWEEN CONTROL AND EXPERIMENTAL GROUPS.

The control condition shows stable performance across runs, with recall values ranging from 0.708 to 0.727 and precision values between 0.627 and 0.639. In contrast, the experimental condition (UCR27) achieves a higher recall value (0.821), indicating improved coverage of security requirements, while presenting a lower precision (0.548), suggesting a higher proportion of non-relevant requirements compared to the oracle.

B. System Usability Scale (SUS) Results

Table III presents the SUS responses obtained from participants interacting with the virtual agent in the experimental condition (UCR27).

Participant	SUS Score
1	55
2	82.5
3	70
4	70
Mean	69.375

TABLE III
SYSTEM USABILITY SCALE (SUS) RESULTS FOR UCR27

The SUS results indicate a mean score of 69.375 suggesting that participants generally perceived the virtual agent as usable and acceptable, although with variability across users.

VI. DISCUSSION

The results of this pilot study provide a descriptive comparison between a fully automated LLM-based approach (UCR26a–UCR26d) and a human-in-the-loop approach supported by a virtual agent (UCR27).

Regarding coverage (recall), the experimental condition (UCR27) obtained a higher value (0.821) compared to the individual control runs, which ranged between 0.708 and 0.727. This indicates that the human-in-the-loop configuration produced a larger number of security requirements relative to the oracle.

In contrast, precision values were lower in the experimental condition (0.548) when compared to the control runs, which remained relatively stable between 0.627 and 0.639. This suggests that the experimental setup generated a higher proportion of requirements not included in the oracle.

The SUS results for the experimental condition show a mean score of 69.375. Individual responses varied between participants, indicating differences in perceived usability of the virtual agent.

VII. CONCLUSION

This work presented a pilot study evaluating a human-in-the-loop approach for the elicitation of software security requirements using an intelligent virtual agent.

The descriptive analysis shows that the experimental condition (UCR27) achieved higher coverage but lower precision compared to the control condition (UCR26a–UCR26d). Additionally, the usability evaluation using the SUS instrument resulted in a mean score of 69.375 for the experimental condition.

These results suggest observable differences between both approaches in this controlled setting. However, given the exploratory nature of the study, the small sample size, and the absence of statistical testing, the findings should be interpreted as preliminary descriptive evidence.

Future work will include additional participants, repeated runs, and inferential statistical analysis to further evaluate the observed trends.

REFERENCES

- [1] M. Riaz, J. King, J. Slankas, and L. Williams, "Hidden in plain sight: Automatically identifying security requirements from natural language artifacts," in *2014 IEEE 22nd International Requirements Engineering Conference (RE)*. IEEE, 2014, pp. 183–192. [Online]. Available: <http://ieeexplore.ieee.org/document/6912260/>
- [2] M. Riaz, J. King, J. Slankas, L. Williams, F. Massacci, C. Quesada-López, and M. Jenkins, "Identifying the implied: Findings from three differentiated replications on the use of security requirements templates," vol. 22, no. 4, pp. 2127–2178, 2017. [Online]. Available: <http://link.springer.com/10.1007/s10664-016-9481-1>
- [3] M. Riaz, J. Slankas, J. King, and L. Williams, "Using templates to elicit implied security requirements from functional requirements - a controlled experiment," in *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. ACM, 2014, pp. 1–10. [Online]. Available: <https://dl.acm.org/doi/10.1145/2652524.2652532>
- [4] M. Riaz and L. Williams, "Security requirements patterns: understanding the science behind the art of pattern writing," in *2012 Second IEEE International Workshop on Requirements Patterns (RePa)*. IEEE, 2012, pp. 29–34. [Online]. Available: <http://ieeexplore.ieee.org/document/6359977/>
- [5] A. Martinez, M. Jenkins, and C. Quesada-Lopez, "Identifying implied security requirements from functional requirements," in *2019 14th Iberian Conference on Information Systems and Technologies (CISTI)*. IEEE, 2019, pp. 1–7. [Online]. Available: <https://ieeexplore.ieee.org/document/8760631/>
- [6] D. Pérez-Morera and C. Quesada-López, "Llm-supported identification of implicit security requirements from functional requirements: An empirical study," 2026, pp. 1–10.
- [7] D. Hybertson, P. Sommerlad, M. Schumacher, F. Buschmann, and E. Fernandez-Buglioni, *Security patterns: integrating security and systems engineering*. Wiley, 2013, OCLC: 865013622.
- [8] M. Riaz, "Inferring security requirements from natural language requirements artifacts." 2016. [Online]. Available: <http://www.lib.ncsu.edu/resolver/1840.16/11378>
- [9] C. B. Haley, J. D. Moffett, R. Laney, and B. Nuseibeh, "A framework for security requirements engineering," in *Proceedings of the 2006 international workshop on Software engineering for secure systems*. ACM, 2006, pp. 35–42. [Online]. Available: <https://dl.acm.org/doi/10.1145/1137627.1137634>
- [10] P. Salini and S. Kanmani, "Elicitation of security requirements for e-health system by applying model oriented security requirements engineering (MOSRE) framework," in *Proceedings of the Second International Conference on Computational Science, Engineering and Information Technology*. ACM, 2012, pp. 126–131. [Online]. Available: <https://dl.acm.org/doi/10.1145/2393216.2393238>
- [11] N. R. Mead and T. Stehney, "Security quality requirements engineering (SQUARE) methodology," vol. 30, no. 4, pp. 1–7, 2005. [Online]. Available: <https://dl.acm.org/doi/10.1145/1082983.1083214>
- [12] C. Haley, R. Laney, J. Moffett, and B. Nuseibeh, "Security requirements engineering: A framework for representation and analysis," vol. 34, no. 1, pp. 133–153, 2008. [Online]. Available: <http://ieeexplore.ieee.org/document/4359475/>
- [13] I. Alexander, "Misuse cases: use cases with hostile intent," vol. 20, no. 1, pp. 58–66, 2003. [Online]. Available: <http://ieeexplore.ieee.org/document/1159030/>
- [14] G. Sindre and A. Opdahl, "Eliciting security requirements by misuse cases," in *Proceedings 37th International Conference on Technology of Object-Oriented Languages and Systems. TOOLS-Pacific 2000*. IEEE Comput. Soc, 2000, pp. 120–131. [Online]. Available: <http://ieeexplore.ieee.org/document/891363/>
- [15] F. A. Braz, E. B. Fernandez, and M. VanHilst, "Eliciting security requirements through misuse activities," in *2008 19th International Conference on Database and Expert Systems Applications*. IEEE, 2008, pp. 328–333. [Online]. Available: <http://ieeexplore.ieee.org/document/4624737/>
- [16] J. McDermott and C. Fox, "Using abuse case models for security requirements analysis," in *Proceedings 15th Annual Computer Security Applications Conference (ACSAC'99)*. IEEE Comput. Soc, 1999, pp. 55–64. [Online]. Available: <http://ieeexplore.ieee.org/document/816013/>
- [17] D. Firesmith, "Specifying reusable security requirements," vol. 3, no. 1, p. 61, 2014. [Online]. Available: http://www.jot.fm/contents/issue_2004_01/column6.html
- [18] A. Toval, J. Nicolás, B. Moros, and F. García, "Requirements reuse for improving information systems security: A practitioner's approach," vol. 6, no. 4, pp. 205–219, 2002. [Online]. Available: <http://link.springer.com/10.1007/PL00010360>
- [19] C. Zhang and D. Budgen, "What do we know about the effectiveness of software design patterns?" vol. 38, no. 5, pp. 1213–1231, 2012. [Online]. Available: <http://ieeexplore.ieee.org/document/5975176/>
- [20] K. Yskout, R. Scandariato, and W. Joosen, "Do security patterns really help designers?" in *Proceedings of the 37th International Conference on Software Engineering - Volume 1*, ser. ICSE '15. IEEE Press, 2015, p. 292–302.
- [21] M. De Gramatica, K. Labunets, F. Massacci, F. Paci, and A. Tedeschi, "The role of catalogues of threats and security controls in security risk assessment: An empirical study with ATM professionals," in *Requirements Engineering: Foundation for Software Quality*, S. A. Fricker and K. Schneider, Eds. Springer International Publishing, 2015, vol. 9013, pp. 98–114, series Title: Lecture Notes in

Computer Science. [Online]. Available: https://link.springer.com/10.1007/978-3-319-16101-3_7

- [22] S. Fedorov, "Ai-driven multi-agent system for real-time security analysis of software releases," in *2025 2nd International Conference on Intelligent Computing and Robotics (ICICR)*, 2025, pp. 1195–1199.
- [23] S. AbdElRahim, S. Ghoniemy, and M. AbouRizka, "Llm-powered network requirement engineering conversational agent for automated 6g datacenter network configuration," in *2025 International Conference on Computer and Applications (ICCA)*, 2025, pp. 1–6.
- [24] J. Cassell, J. Sullivan, S. Prevost, and E. F. Churchill, Eds., *Embodied Conversational Agents*. The MIT Press, Mar. 2000. [Online]. Available: <https://direct.mit.edu/books/book/3227/Embodied-Conversational-Agents>
- [25] T. W. Bickmore and R. W. Picard, "Establishing and maintaining long-term human-computer relationships," *ACM Trans. Comput.-Hum. Interact.*, vol. 12, no. 2, p. 293–327, Jun. 2005. [Online]. Available: <https://doi.org/10.1145/1067860.1067867>
- [26] N. Yee and J. Bailenson, "The Proteus Effect: The Effect of Transformed Self-Representation on Behavior," *Human Communication Research*, vol. 33, no. 3, pp. 271–290, Jul. 2007. [Online]. Available: <https://academic.oup.com/hcr/article/33/3/271-290/4210718>